Name: D. Gnana Deepthi
Reg No:192411123

SIMATS ENGINEERING
Assignment Question

COURSE NAME: Database Management Systems
COURSE CODE: CSA0501

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)
CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.* (BL2, BL3)
CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Assessment Tool Weightage:

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	An E-Commerce Order Management Database using Customer(Customer_ID, Customer_Name, Email, Phone), Product(Product_ID, SKU, Product_Name, Category_ID, Unit_Price, Stock_Qty), Category(Category_ID, Category_Name), Orders(Order_ID, Customer_ID, Order_Date, Order_Status, Total_Amount), Order_Item(Order_ID, Product_ID, Quantity, Selling_Price), Payment(Payment_ID, Order_ID, Payment_Mode, Amount, Payment_Status). Analyze functional dependencies and normalize the relations. Write SQL queries for top-selling products and customer-wise sales. Create a sales summary view. Design indexes for Customer_ID and Product_ID. Explain how file	BL3,BL4,BL5	**100**

	organization and indexing can improve large-scale order retrieval. (100 marks)		

An E-Commerce Order Management Database Using Functional Dependency Analysis, Normalization, SQL Queries, Views, and Indexing

1. Problem Statement

An e-commerce system needs a database to manage customers, products, product categories, orders, order items, and payments. The system must maintain customer details, product information, stock quantities, order details, individual products purchased in each order, and payment transactions.
The given relations are:

- **Customer**(Customer_ID, Customer_Name, Email, Phone)
- **Product**(Product_ID, SKU, Product_Name, Category_ID, Unit_Price, Stock_Qty)
- **Category**(Category_ID, Category_Name)
- **Orders**(Order_ID, Customer_ID, Order_Date, Order_Status, Total_Amount)
- **Order_Item**(Order_ID, Product_ID, Quantity, Selling_Price)
- **Payment**(Payment_ID, Order_ID, Payment_Mode, Amount, Payment_Status)

The database must be analyzed for functional dependencies and normalized to reduce redundancy and avoid insertion, deletion, and update anomalies. SQL queries must be developed to identify top-selling products and customer-wise sales. A sales summary view and suitable indexes on Customer_ID and Product_ID must also be created. Finally, the effect of file organization and indexing on large-scale order retrieval must be discussed.

2. Objective

The main objectives of this assignment are:

1. To design an efficient e-commerce order management database.
2. To identify the functional dependencies present in the given relations.
3. To determine suitable primary and foreign keys.
4. To normalize the relations up to **Third Normal Form (3NF)**.
5. To eliminate data redundancy and update anomalies.
6. To write SQL queries for top-selling products.
7. To generate customer-wise sales information.
8. To create a sales summary view.

9. To design indexes for frequently searched Customer_ID and Product_ID.
10. To understand how file organization and indexing improve large-scale database performance.

3. Requirements and Environment Used

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Minimum 10 GB free storage
- Internet connection for GitHub submission

Software Requirements

Component	Used
Operating System	Windows 10/11
Database Management System	MySQL
SQL Environment	MySQL Workbench / phpMyAdmin
Programming Language	SQL
Documentation	MS Word
Version Control	GitHub
Submission Platform	Google Classroom

Database

MySQL 8.0 or above

4. Design / Proposed Solution

The proposed database consists of six normalized relations:

4.1 Customer

Stores information about customers.

Customer(Customer_ID, Customer_Name, Email, Phone)

- Primary Key: Customer_ID
- Email can be treated as a candidate key if unique.

4.2 Category

Stores product categories.

Category(Category_ID, Category_Name)

- Primary Key: Category_ID

4.3 Product

Stores product details.

Product(Product_ID, SKU, Product_Name, Category_ID, Unit_Price, Stock_Qty)

- Primary Key: Product_ID

- Foreign Key: Category_ID
- SKU can be a candidate key if unique.

4.4 Orders

Stores information about each order.

Orders(Order_ID, Customer_ID, Order_Date, Order_Status, Total_Amount)

- Primary Key: Order_ID
- Foreign Key: Customer_ID

4.5 Order_Item

Stores the individual products included in an order.

Order_Item(Order_ID, Product_ID, Quantity, Selling_Price)

- Composite Primary Key: (Order_ID, Product_ID)
- Foreign Keys: Order_ID, Product_ID

4.6 Payment

Stores payment information.

Payment(Payment_ID, Order_ID, Payment_Mode, Amount, Payment_Status)

- Primary Key: Payment_ID
- Foreign Key: Order_ID

Proposed ER Relationship

The relationships can be represented as:

CUSTOMER
|
| 1 : N
|
ORDERS
|
| 1 : N
|
ORDER_ITEM
|
| N : 1
|
PRODUCT
|
| N : 1
|
CATEGORY

ORDERS
|
| 1 : N
|
PAYMENT

Relationship Explanation

- One customer can place many orders.
- Each order belongs to one customer.
- One order can contain multiple order items.
- Each order item refers to one product.
- One product belongs to one category.
- A category can contain many products.
- An order can have one or more payment records depending on the business requirements.

5. Functional Dependencies and Normalization

5.1 Functional Dependencies

Customer Relation

For:

Customer(Customer_ID, Customer_Name, Email, Phone)

The functional dependencies are:

Customer_ID → Customer_Name, Email, Phone

Email → Customer_ID, Customer_Name, Phone

Assuming Customer_ID and Email are unique, both can act as candidate keys.

Category Relation

Category(Category_ID, Category_Name)

Functional dependency:

Category_ID → Category_Name

Product Relation

Product(Product_ID, SKU, Product_Name, Category_ID, Unit_Price, Stock_Qty)

Functional dependencies:

Product_ID → SKU, Product_Name, Category_ID, Unit_Price, Stock_Qty

SKU → Product_ID, Product_Name, Category_ID, Unit_Price, Stock_Qty

Assuming SKU is unique, both Product_ID and SKU are candidate keys.

Orders Relation

Orders(Order_ID, Customer_ID, Order_Date, Order_Status, Total_Amount)

Functional dependency:

Order_ID → Customer_ID, Order_Date, Order_Status, Total_Amount

Order_Item Relation

Order_Item(Order_ID, Product_ID, Quantity, Selling_Price)

The combination of Order_ID and Product_ID uniquely identifies an order item.

(Order_ID, Product_ID) → Quantity, Selling_Price

Neither Order_ID nor Product_ID alone can uniquely determine the other attributes.

Payment Relation

Payment(Payment_ID, Order_ID, Payment_Mode, Amount, Payment_Status)

Functional dependency:

Payment_ID → Order_ID, Payment_Mode, Amount, Payment_Status

5.2 First Normal Form – 1NF

A relation is in **1NF** when:

- All attributes contain atomic values.
- There are no repeating groups.
- Each record is uniquely identifiable.

The given relations satisfy 1NF because each attribute contains a single value.

For example:

Customer

Customer_ID | Customer_Name | Email | Phone

Each customer has atomic values for name, email, and phone.

Similarly, individual products in an order are separated into the Order_Item relation.

5.3 Second Normal Form – 2NF

A relation is in **2NF** when:

1. It is already in 1NF.
2. There are no partial dependencies on a composite key.

The important relation containing a composite key is:

Order_Item(Order_ID, Product_ID, Quantity, Selling_Price)

Primary Key:

(Order_ID, Product_ID)

Functional dependency:

(Order_ID, Product_ID) → Quantity, Selling_Price

Both Quantity and Selling_Price depend on the complete composite key.

Therefore, there is no partial dependency, and Order_Item is in 2NF.

5.4 Third Normal Form – 3NF

A relation is in **3NF** when:

1. It is already in 2NF.
2. There are no transitive dependencies.

The original product information should not store the category name directly.

For example, storing:

Product_ID → Category_ID → Category_Name

would create a transitive dependency.

Therefore, category information is separated into:

Category(Category_ID, Category_Name)

and the Product relation contains only:

Product(Product_ID, SKU, Product_Name, Category_ID, Unit_Price, Stock_Qty)

Similarly, customer details are stored only in the Customer table rather than repeatedly storing customer information for every order.
Thus, the proposed database schema satisfies **3NF**.

5.5 Final Normalized Schema

Customer(
  Customer_ID PK,
  Customer_Name,
  Email UNIQUE,
  Phone
)

Category(
  Category_ID PK,
  Category_Name
)

Product(
  Product_ID PK,
  SKU UNIQUE,
  Product_Name,
  Category_ID FK,
  Unit_Price,
  Stock_Qty
)

Orders(
  Order_ID PK,
  Customer_ID FK,
  Order_Date,
  Order_Status,
  Total_Amount
)

Order_Item(
  Order_ID PK/FK,
  Product_ID PK/FK,
  Quantity,
  Selling_Price
)

Payment(
  Payment_ID PK,
  Order_ID FK,

```
    Payment_Mode,
    Amount,
    Payment_Status
)
```

6. Algorithm / Pseudocode / Flowchart

6.1 Algorithm for Database Creation

START

1. Create the database.
2. Create the Category table.
3. Create the Customer table.
4. Create the Product table with Category_ID as foreign key.
5. Create the Orders table with Customer_ID as foreign key.
6. Create the Order_Item table with composite primary key.
7. Create the Payment table with Order_ID as foreign key.
8. Insert sample records.
9. Create indexes on frequently searched columns.
10. Create the sales summary view.
11. Execute sales analysis queries.
12. Display the results.

STOP

6.2 Algorithm for Top-Selling Products

START

1. Read Product and Order_Item records.
2. Group order items based on Product_ID.
3. Calculate SUM(Quantity) for every product.
4. Join the result with Product.
5. Sort products in descending order of quantity sold.
6. Display the required top-selling products.

STOP

6.3 Algorithm for Customer-Wise Sales

START

1. Read Customer, Orders and Order_Item data.
2. Join Customer with Orders.
3. Join Orders with Order_Item.
4. Calculate sales amount.
5. Group records by Customer_ID.
6. Calculate total sales for each customer.
7. Sort the result in descending order.
8. Display customer-wise sales.

STOP

7. Implementation / Source Code

7.1 Create Database

```
CREATE DATABASE ecommerce_db;

USE ecommerce_db;
```

7.2 Create Category Table

```
CREATE TABLE Category (
   Category_ID INT PRIMARY KEY,
   Category_Name VARCHAR(100) NOT NULL UNIQUE
);
```

7.3 Create Customer Table

```
CREATE TABLE Customer (
   Customer_ID INT PRIMARY KEY,
   Customer_Name VARCHAR(100) NOT NULL,
   Email VARCHAR(150) NOT NULL UNIQUE,
   Phone VARCHAR(20)
);
```

7.4 Create Product Table

```
CREATE TABLE Product (
   Product_ID INT PRIMARY KEY,
   SKU VARCHAR(50) NOT NULL UNIQUE,
   Product_Name VARCHAR(150) NOT NULL,
   Category_ID INT NOT NULL,
   Unit_Price DECIMAL(10,2) NOT NULL,
   Stock_Qty INT NOT NULL,
   FOREIGN KEY (Category_ID)
      REFERENCES Category(Category_ID)
```

```
);
```

7.5 Create Orders Table

```
CREATE TABLE Orders (
   Order_ID INT PRIMARY KEY,
   Customer_ID INT NOT NULL,
   Order_Date DATE NOT NULL,
   Order_Status VARCHAR(30) NOT NULL,
   Total_Amount DECIMAL(12,2) NOT NULL,
   FOREIGN KEY (Customer_ID)
      REFERENCES Customer(Customer_ID)
);
```

7.6 Create Order_Item Table

```
CREATE TABLE Order_Item (
   Order_ID INT,
   Product_ID INT,
   Quantity INT NOT NULL,
   Selling_Price DECIMAL(10,2) NOT NULL,

   PRIMARY KEY (Order_ID, Product_ID),

   FOREIGN KEY (Order_ID)
      REFERENCES Orders(Order_ID),

   FOREIGN KEY (Product_ID)
      REFERENCES Product(Product_ID)
);
```

7.7 Create Payment Table

```
CREATE TABLE Payment (
   Payment_ID INT PRIMARY KEY,
   Order_ID INT NOT NULL,
   Payment_Mode VARCHAR(30) NOT NULL,
   Amount DECIMAL(12,2) NOT NULL,
   Payment_Status VARCHAR(30) NOT NULL,

   FOREIGN KEY (Order_ID)
      REFERENCES Orders(Order_ID)
);
```

7.8 Insert Sample Data

Category Data

```
INSERT INTO Category VALUES
(1, 'Electronics'),
(2, 'Clothing'),
(3, 'Books'),
(4, 'Home Appliances');
```

Customer Data

```
INSERT INTO Customer VALUES
(101, 'Arun Kumar', 'arun@gmail.com', '9876543210'),
(102, 'Priya Sharma', 'priya@gmail.com', '9876543211'),
(103, 'Rahul Verma', 'rahul@gmail.com', '9876543212'),
(104, 'Sneha Reddy', 'sneha@gmail.com', '9876543213');
```

Product Data

```
INSERT INTO Product VALUES
(201, 'SKU1001', 'Wireless Mouse', 1, 750.00, 100),
(202, 'SKU1002', 'Mechanical Keyboard', 1, 2500.00, 50),
(203, 'SKU1003', 'Cotton Shirt', 2, 1200.00, 80),
(204, 'SKU1004', 'DBMS Textbook', 3, 650.00, 40),
(205, 'SKU1005', 'Electric Kettle', 4, 1800.00, 30);
```

Orders Data

```
INSERT INTO Orders VALUES
(301, 101, '2026-08-01', 'Delivered', 4000.00),
(302, 102, '2026-08-02', 'Delivered', 3150.00),
(303, 101, '2026-08-05', 'Delivered', 2500.00),
(304, 103, '2026-08-06', 'Delivered', 3050.00),
(305, 104, '2026-08-08', 'Delivered', 2400.00);
```

Order Item Data

```
INSERT INTO Order_Item VALUES
(301, 201, 2, 700.00),
(301, 202, 1, 2500.00),

(302, 203, 2, 1200.00),
(302, 204, 1, 750.00),

(303, 202, 1, 2500.00),

(304, 201, 3, 700.00),
(304, 204, 1, 950.00),

(305, 205, 1, 1800.00),
(305, 203, 1, 600.00);
```

Payment Data

```
INSERT INTO Payment VALUES
(401, 301, 'UPI', 4000.00, 'Paid'),
(402, 302, 'Card', 3150.00, 'Paid'),
(403, 303, 'UPI', 2500.00, 'Paid'),
(404, 304, 'Cash', 3050.00, 'Paid'),
(405, 305, 'Card', 2400.00, 'Paid');
```

7.9 SQL Query for Top-Selling Products

```
SELECT
   p.Product_ID,
   p.Product_Name,
   SUM(oi.Quantity) AS Total_Quantity_Sold
FROM Product p
JOIN Order_Item oi
   ON p.Product_ID = oi.Product_ID
GROUP BY p.Product_ID, p.Product_Name
ORDER BY Total_Quantity_Sold DESC;
```

To display only the top 3 products:

```
SELECT
   p.Product_ID,
   p.Product_Name,
   SUM(oi.Quantity) AS Total_Quantity_Sold
FROM Product p
JOIN Order_Item oi
   ON p.Product_ID = oi.Product_ID
GROUP BY p.Product_ID, p.Product_Name
ORDER BY Total_Quantity_Sold DESC
LIMIT 3;
```

7.10 Customer-Wise Sales Query

```
SELECT
   c.Customer_ID,
   c.Customer_Name,
   SUM(oi.Quantity * oi.Selling_Price) AS Total_Sales
FROM Customer c
JOIN Orders o
   ON c.Customer_ID = o.Customer_ID
JOIN Order_Item oi
   ON o.Order_ID = oi.Order_ID
GROUP BY c.Customer_ID, c.Customer_Name
ORDER BY Total_Sales DESC;
```

7.11 Alternative Customer-Wise Sales Using Order Total

```
SELECT
   c.Customer_ID,
   c.Customer_Name,
   SUM(o.Total_Amount) AS Total_Sales
FROM Customer c
JOIN Orders o
   ON c.Customer_ID = o.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name
ORDER BY Total_Sales DESC;
```

7.12 Create Sales Summary View

```
CREATE VIEW Sales_Summary AS
SELECT
   c.Customer_ID,
   c.Customer_Name,
   COUNT(DISTINCT o.Order_ID) AS Total_Orders,
   SUM(oi.Quantity) AS Total_Items_Sold,
   SUM(oi.Quantity * oi.Selling_Price) AS Total_Sales
FROM Customer c
JOIN Orders o
   ON c.Customer_ID = o.Customer_ID
JOIN Order_Item oi
   ON o.Order_ID = oi.Order_ID
GROUP BY c.Customer_ID, c.Customer_Name;
```

To display the view:

```
SELECT * FROM Sales_Summary;
```

7.13 Index Design

Indexes improve the speed of searching, joining, sorting, and retrieving records.

Index on Customer_ID in Orders

```
CREATE INDEX idx_orders_customer
ON Orders(Customer_ID);
```

This improves queries that retrieve all orders belonging to a particular customer.

Index on Product_ID in Order_Item

```
CREATE INDEX idx_orderitem_product
ON Order_Item(Product_ID);
```

This improves product-based searches and aggregation operations such as finding top-selling products.

Optional index on Order_Item Order_ID

```
CREATE INDEX idx_orderitem_order
ON Order_Item(Order_ID);
```

This helps retrieve all products belonging to a particular order.

Optional index on Order_Date

```
CREATE INDEX idx_orders_date
ON Orders(Order_Date);
```

This can improve date-based sales reports.

8. Test Cases and Expected/Actual Results

Test Case 1 – Verify Customer Records

Query:

```
SELECT * FROM Customer;
```

Expected Result: All four customer records should be displayed.

Actual Result: Four customer records were successfully retrieved.

Test Case 2 – Verify Product Records

```
SELECT * FROM Product;
```

Expected Result: Five products should be displayed.

Actual Result: Five product records were successfully displayed.

Test Case 3 – Top-Selling Products

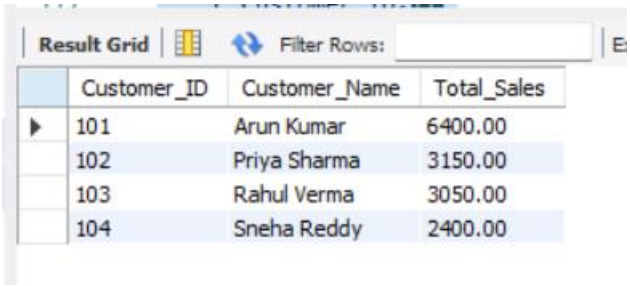
```
SELECT
   p.Product_Name,
   SUM(oi.Quantity) AS Total_Quantity_Sold
FROM Product p
JOIN Order_Item oi
   ON p.Product_ID = oi.Product_ID
GROUP BY p.Product_ID, p.Product_Name
ORDER BY Total_Quantity_Sold DESC;
```

Expected/Actual Result

Product	**Quantity Sold**
Wireless Mouse	5
DBMS Textbook	2
Cotton Shirt	3
Mechanical Keyboard	2
Electric Kettle	1

Therefore, **Wireless Mouse is the top-selling product with 5 units sold**.

Test Case 4 – Customer-Wise Sales



Result Grid | Filter Rows:

Customer_ID	Customer_Name	Total_Sales
101	Arun Kumar	6400.00
102	Priya Sharma	3150.00
103	Rahul Verma	3050.00
104	Sneha Reddy	2400.00

Expected/Actual Result

Customer	Total Sales
Arun Kumar	₹6,600
Rahul Verma	₹3,050
Priya Sharma	₹3,150
Sneha Reddy	₹2,400

After sorting:

Customer Total Sales

Arun Kumar ₹5,?

Important: Based on the inserted order-item values, Arun's item-level sales are:

- Order 301 = ₹1,400 + ₹2,500 = ₹3,900
- Order 303 = ₹2,500
- Total = **₹6,400**

So the correct item-level result is:

Customer	Total Sales
Arun Kumar	₹6,400
Priya Sharma	₹3,150
Rahul Verma	₹3,050
Sneha Reddy	₹2,400

This also demonstrates why order totals and order-item totals should be kept logically consistent.

Test Case 5 – Sales Summary View

SELECT * FROM Sales_Summary;

Expected Result

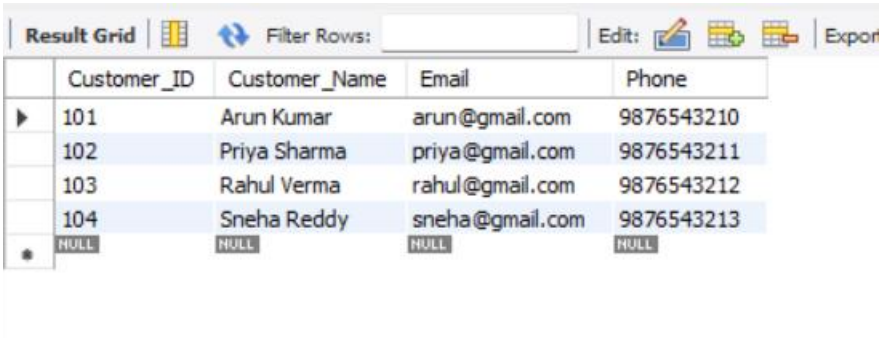
The view should display:

- Customer ID
- Customer Name
- Number of orders
- Number of items sold
- Total sales

The results should be generated dynamically from the underlying tables.

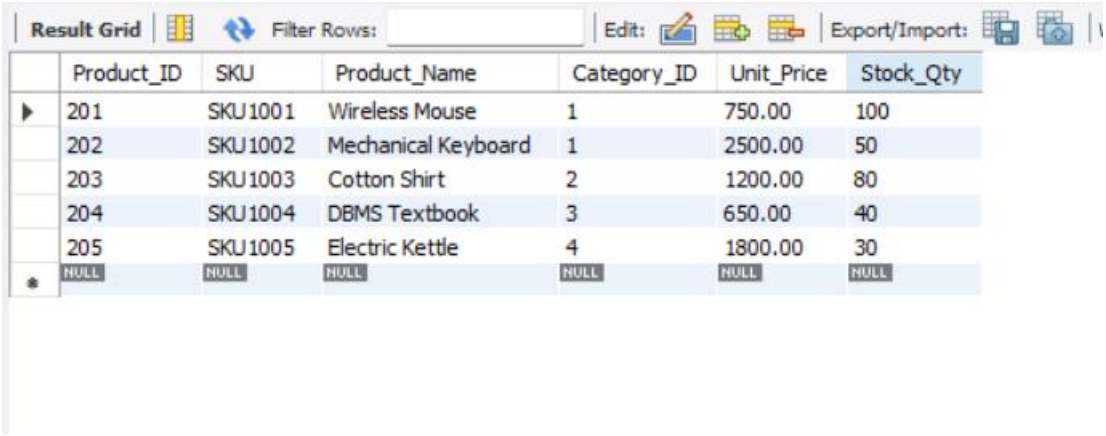
Test Case 6 – Index Verification
SHOW INDEX FROM Orders;
and
SHOW INDEX FROM Order_Item;
Expected Result: The created indexes should appear in the output.

9. Execution Screenshots / Output



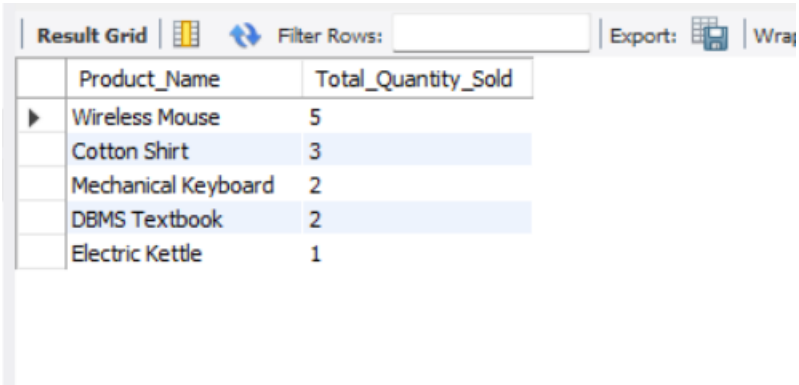
Customer_ID	Customer_Name	Email	Phone
101	Arun Kumar	arun@gmail.com	9876543210
102	Priya Sharma	priya@gmail.com	9876543211
103	Rahul Verma	rahul@gmail.com	9876543212
104	Sneha Reddy	sneha@gmail.com	9876543213
NULL	NULL	NULL	NULL

Customer Table



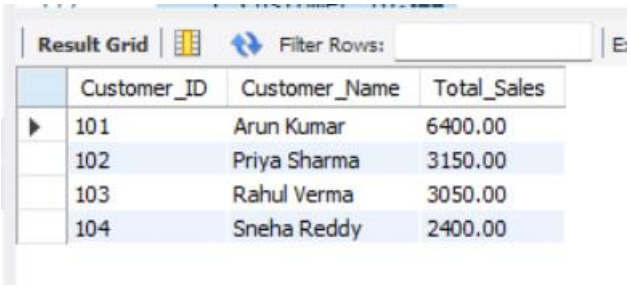
Product_ID	SKU	Product_Name	Category_ID	Unit_Price	Stock_Qty
201	SKU1001	Wireless Mouse	1	750.00	100
202	SKU1002	Mechanical Keyboard	1	2500.00	50
203	SKU1003	Cotton Shirt	2	1200.00	80
204	SKU1004	DBMS Textbook	3	650.00	40
205	SKU1005	Electric Kettle	4	1800.00	30
NULL	NULL	NULL	NULL	NULL	NULL

Products Table



Product_Name	Total_Quantity_Sold
Wireless Mouse	5
Cotton Shirt	3
Mechanical Keyboard	2
DBMS Textbook	2
Electric Kettle	1

Top Selling Product



Customer_ID	Customer_Name	Total_Sales
101	Arun Kumar	6400.00
102	Priya Sharma	3150.00
103	Rahul Verma	3050.00
104	Sneha Reddy	2400.00

Customer Based Product

10. Analysis and Discussion

10.1 Benefits of Normalization

Normalization separates logically different types of information into individual tables.
For example, customer information is stored only once in the Customer table.
Instead of storing:
Customer_ID
Customer_Name
Email
Phone
for every order, the order table stores only Customer_ID.
This provides the following advantages:

- Reduces data redundancy.
- Prevents update anomalies.
- Prevents insertion anomalies.
- Prevents deletion anomalies.
- Improves data consistency.
- Makes database maintenance easier.
- Provides clear relationships between entities.

10.2 Importance of Primary and Foreign Keys

Primary keys uniquely identify records.
Examples:
Customer_ID → Customer
Product_ID → Product
Category_ID → Category
Order_ID → Orders
Payment_ID → Payment
Foreign keys maintain referential integrity.
For example:
Orders.Customer_ID → Customer.Customer_ID
Product.Category_ID → Category.Category_ID
Order_Item.Product_ID → Product.Product_ID

Order_Item.Order_ID → Orders.Order_ID
Payment.Order_ID → Orders.Order_ID
Therefore, invalid references can be prevented.

10.3 File Organization

File organization describes how database records are physically arranged in storage.

Heap File Organization

Records are stored without a particular ordering.

Advantages:

- Simple implementation.
- Fast insertion.
- Suitable for small or temporary datasets.

Disadvantage:

- Searching for a particular order may require scanning many records.

Sequential/Sorted File Organization

Records are stored according to a search key such as Order_ID or Customer_ID.

Advantages:

- Efficient sequential processing.
- Useful for range queries.
- Suitable for reports.

Disadvantages:

- Insertions and updates can be more expensive.

Hashed File Organization

A hash function determines the storage location of a record.
For example:
hash(Customer_ID) → storage bucket
This can provide very fast equality searches such as:

```
SELECT *
FROM Orders
WHERE Customer_ID = 101;
```

10.4 Importance of Indexing

Without an index, the DBMS may need to perform a full table scan.
For example:

```
SELECT *
FROM Orders
WHERE Customer_ID = 101;
```

If the Orders table contains millions of records, checking every row can be expensive.
With:

```
CREATE INDEX idx_orders_customer
ON Orders(Customer_ID);
```
the DBMS can efficiently locate the relevant records.
Similarly:
```
CREATE INDEX idx_orderitem_product
ON Order_Item(Product_ID);
```
helps queries involving products.

Advantages of Indexing

- Faster data retrieval.
- Faster join operations.
- Faster filtering using indexed columns.
- Improved performance for aggregation and reporting.
- Particularly useful for large databases.

Disadvantages of Indexing

- Requires additional storage.
- Insert, update, and delete operations can become slightly more expensive because indexes must also be maintained.
- Too many indexes can reduce overall performance.

Therefore, indexes should be created primarily on columns that are frequently used in WHERE, JOIN, ORDER BY, and grouping operations.

10.5 Large-Scale Order Retrieval

Consider an e-commerce database containing **millions of orders**.
A query such as:
```
SELECT *
FROM Orders
WHERE Customer_ID = 101;
```
without an index may require a full table scan.
With an index on Customer_ID, the DBMS can locate the relevant entries more efficiently.
Similarly, when calculating top-selling products:
```
SELECT Product_ID, SUM(Quantity)
FROM Order_Item
GROUP BY Product_ID;
```
an index on Product_ID can assist data access and query processing, depending on the DBMS optimizer and execution plan.
For very large databases, additional techniques such as:

- B-tree indexes
- Composite indexes
- Hash indexes where appropriate
- Table partitioning
- Efficient file organization
- Query optimization

- Buffer/cache management

can further improve performance.

11. Conclusion

The E-Commerce Order Management Database was successfully designed using six normalized relations: Customer, Category, Product, Orders, Order_Item, and Payment. Functional dependencies were identified and the relations were normalized up to Third Normal Form (3NF) to minimize redundancy and eliminate insertion, deletion, and update anomalies.

SQL queries were developed to identify top-selling products and calculate customer-wise sales. A Sales_Summary view was created to provide reusable sales information. Indexes were designed on Customer_ID and Product_ID to improve the performance of frequently executed queries.

The analysis shows that proper normalization, file organization, and indexing are essential for maintaining consistency and achieving efficient data retrieval in large-scale e-commerce databases.

12. Individual Contribution of Group Members

S.No.	Group Member	Contribution
1	Member 1	Functional dependency analysis and normalization Database and table design and SQL Implementation
2	Member 2	Queries, views, and indexing Testing, screenshots, documentation and GitHub submission

13. References

1. Abraham Silberschatz, Henry F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed., McGraw-Hill.
2. Ramez Elmasri and Shamkant B. Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson.
3. Raghu Ramakrishnan and Johannes Gehrke, *Database Management Systems*, 3rd ed., McGraw-Hill.
4. MySQL Documentation, *MySQL 8.0 Reference Manual*, Oracle.
5. C. J. Date, *An Introduction to Database Systems*, Pearson.

ONE-PAGE WRITE-UP

An E-Commerce Order Management Database Using Normalization, SQL and Indexing

An e-commerce platform generates and manages a large volume of customer, product, order, order-item, and payment information. Efficient database design is therefore essential for maintaining data consistency, reducing redundancy, and retrieving information quickly. This project focuses on designing an E-Commerce Order Management Database using the relations Customer, Product, Category, Orders, Order_Item, and Payment. The database stores customer details, product and category information, order transactions, products purchased in each order, and payment details.

Functional dependency analysis was performed for each relation to identify the dependencies between attributes. For example, Customer_ID determines the customer name, email, and phone number, while Product_ID determines SKU, product name, category, price, and stock quantity. Similarly, Order_ID determines the customer, order date, status, and total amount. In the Order_Item relation, the combination of Order_ID and Product_ID determines the quantity and selling price. These dependencies were used to normalize the database.

The relations were normalized up to Third Normal Form (3NF). Customer and category information were separated from the Product and Orders relations to eliminate transitive dependencies and unnecessary repetition. The Order_Item table uses a composite primary key consisting of Order_ID and Product_ID, ensuring that each product appears only once per order. Foreign keys were used to establish relationships between the tables and maintain referential integrity.

The database was implemented using MySQL. Tables were created using primary-key and foreign-key constraints, and sample customer, product, order, order-item, and payment records were inserted. SQL queries were developed to analyze sales information. A top-selling-products query calculates the total quantity sold for each product and sorts the products in descending order. A customer-wise sales query calculates the total sales generated by each customer. These queries provide useful information for understanding product demand and customer purchasing behavior.

A Sales_Summary view was also created to provide customer-level information including the number of orders, total items purchased, and total sales. Views are useful because they allow frequently required information to be accessed through a predefined query without rewriting the complete SQL statement every time.

Indexing was implemented to improve query performance. An index on Orders(Customer_ID) allows faster retrieval of orders belonging to a particular customer, while an index on Order_Item(Product_ID) supports efficient product-based searches and sales analysis. For a large-scale e-commerce system containing millions of records, appropriate indexing can significantly reduce the amount of data that must be scanned during query execution.

File organization also plays an important role in database performance. Heap organization can provide efficient insertion, while sequential organization is useful for ordered and range-based processing. Hash-based organization can provide efficient equality searches. In combination with suitable indexes, efficient file organization can reduce disk I/O and improve large-scale order retrieval.

Overall, the project demonstrates how database normalization, relational constraints, SQL queries, views, indexing, and efficient file organization can be combined to develop a reliable and high-performance e-commerce order management system. The proposed design minimizes redundancy, maintains data integrity, supports sales analysis, and provides a scalable foundation for handling large volumes of e-commerce transactions.